

Big Data Modeling & Analytics (course outline)

Mahmoud Parsian

Ph.D. in Computer Science



Big Data Modeling & Analytics?

1. What is Big Data?
2. What is Big data Modeling?
3. What is Big Data Modeling & Analytics?
4. **Solutions** to Big Data Modeling & Analytics
 - **MapReduce paradigm**
 - **Spark/PySpark**
 - **Amazon Athena & Google's Big Query**
 - ...



1. What is Big Data?

Big Data refers to datasets that are so massive, fast-moving, and complex that traditional data-processing software cannot capture, store, or analyze them effectively.

Instead of relying on a single computer, big data requires specialized frameworks like

- Apache Spark
- Cluster Computing
- Snowflake
- Amazon Athena / Google's BigQuery



What is a Big Data Modeling?

Big data modeling is the process of **organizing large**, **fast-moving**, and **diverse datasets** so they can be **stored**, **processed**, **analyzed**, and **understood** efficiently.

Key Characteristics

1. **Volume**: Handles massive amounts of data
2. **Velocity**: Supports rapidly generated and processed data
3. **Variety**: Combines structured, semi-structured, and unstructured data
4. **Veracity**: The quality, reliability, and trust of the data
5. **Scalability**: Adapts as data and user demand grow



What is a Big Data Modeling?

Common Models

- Data lakes and Lake-Houses
- NoSQL models: document, key-value, column, and graph
- Distributed data warehouses (Snowflake)
- Batch and streaming architectures (Spark)

> **Goal:** Turn complex raw data into a reliable structure for analysis and decision-making.



What is a Big Data Modeling & Analytics?

Big data analytics uses modeled data to discover patterns, explain outcomes, predict future events, and recommend actions.

Analytics Types

- Descriptive: What happened?
- Diagnostic: Why did it happen?
- Predictive: What is likely to happen?
- Prescriptive: What action should be taken?

Typical Workflow

Collect → Clean → Model → Process → Analyze → Visualize → Act



Business Value of Big Data Modeling & Analytics?

1. **Faster, evidence-based decisions**
 2. **Improved customer experiences**
 3. **Fraud and risk detection**
 4. **Operational efficiency**
 5. **Forecasting and innovation**
 6. **Building AI-LLM models**
-
- **Strong analytics** depends **on a scalable, accurate,**
 - **and well-governed data model.**

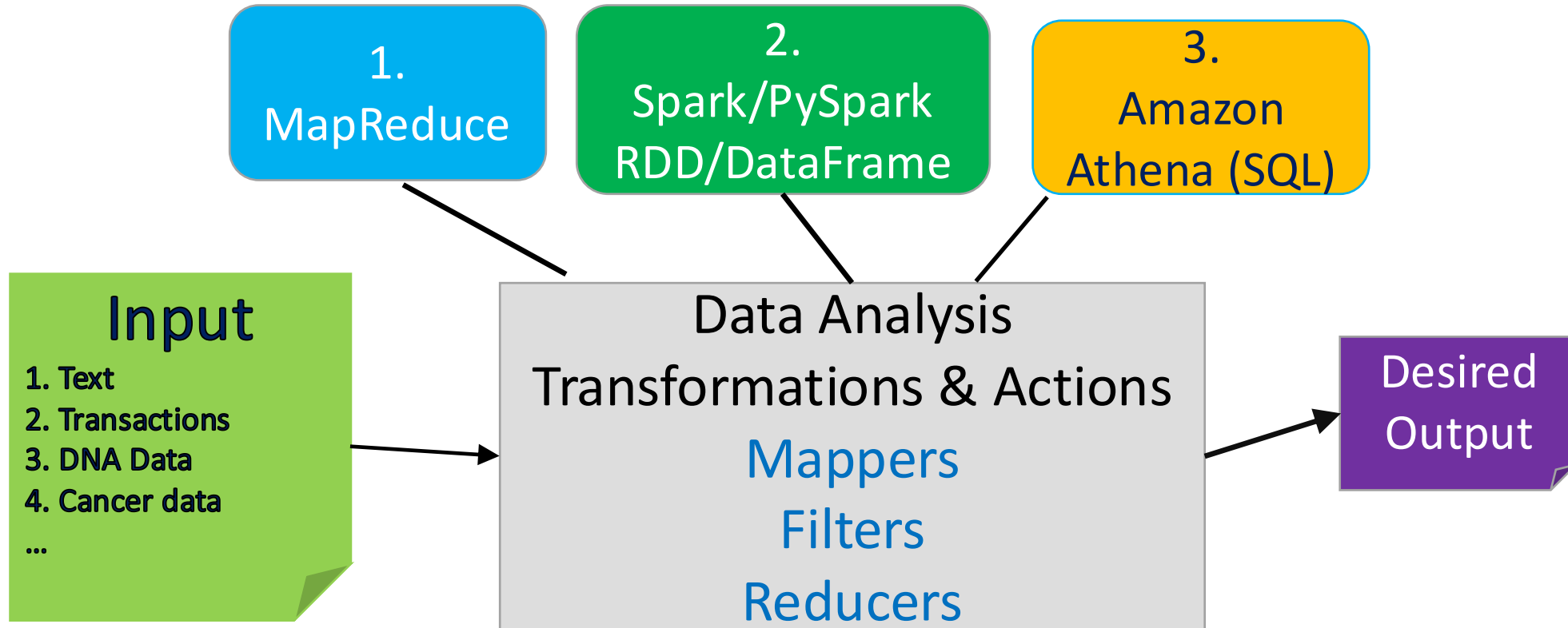


Main Course Components:

1. Introduction to MapReduce Paradigm (25%)
2. Spark and PySpark (60%)
3. Serverless SQL Access to Big Data (15%)



Big Data Analysis

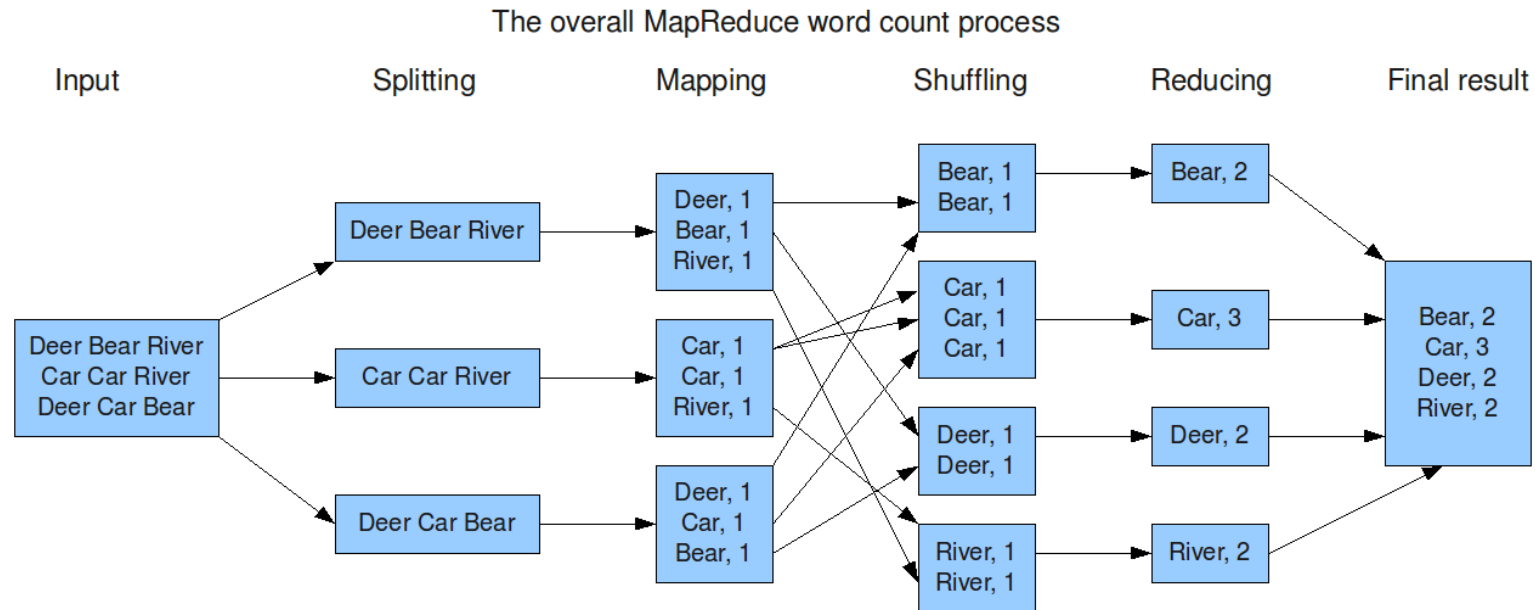


1. MapReduce Paradigm

- **MapReduce** is a programming **paradigm/model** that enables **massive scalability** across hundreds or thousands of servers in a cluster.
- **MapReduce has 3 functions:**
 - `map()`
 - `reduce()`
 - `combine()`



1. MapReduce Paradigm



1. MapReduce Programs

- **For MapReduce, we will use:**
 - Pseudo code (not executable)
 - PySpark code (runs on Spark)
- **For MapReduce, we will focus on learning MapReduce paradigm/model that enables massive scalability across hundreds or thousands of servers in a cluster.**
- **For MapReduce, pseudo-code**
 - examples are given in Lin, J., & Dyer, C. (2010). *Data-intensive text processing with MapReduce*



1. MapReduce Components

MapReduce has 3 functions:

`map()`

- Mapper function

`reduce()`

- Reducer function

`combine()`

- Optional combiner function

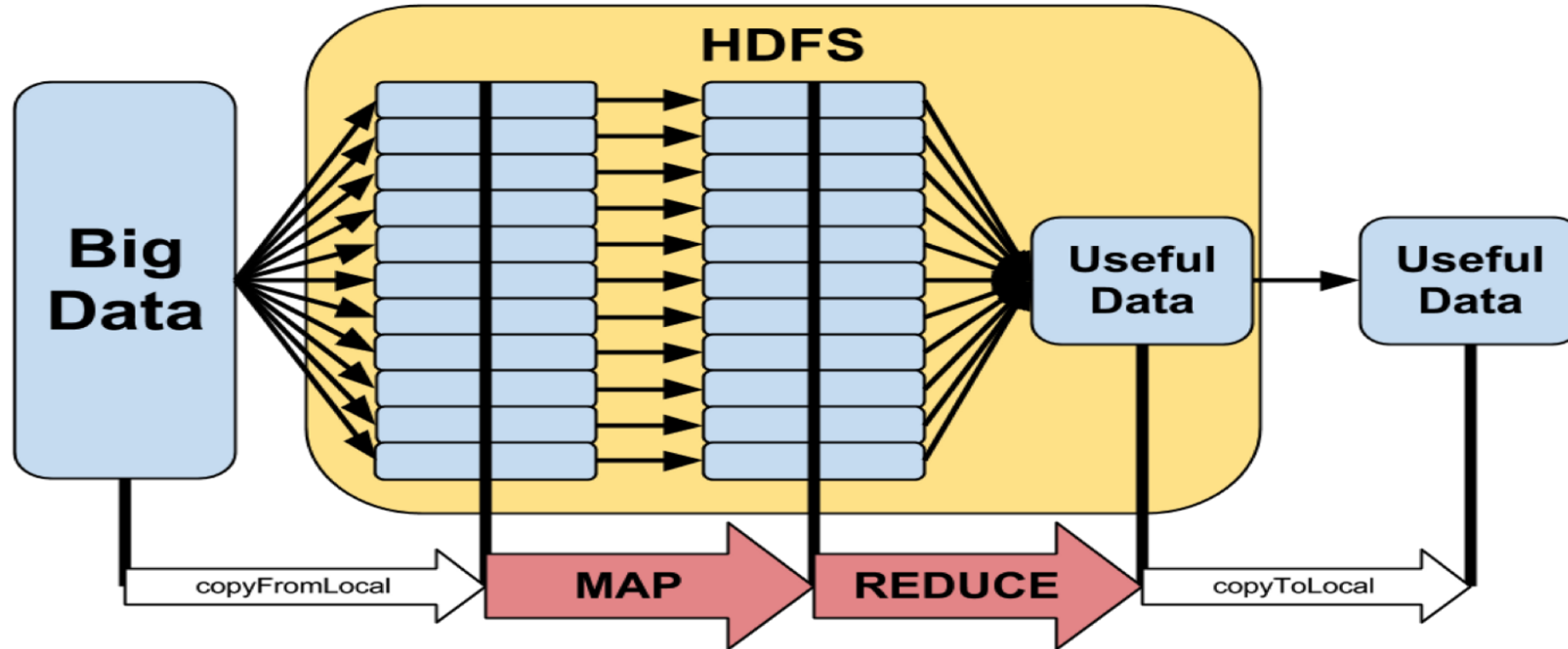


MapReduce Paradigm

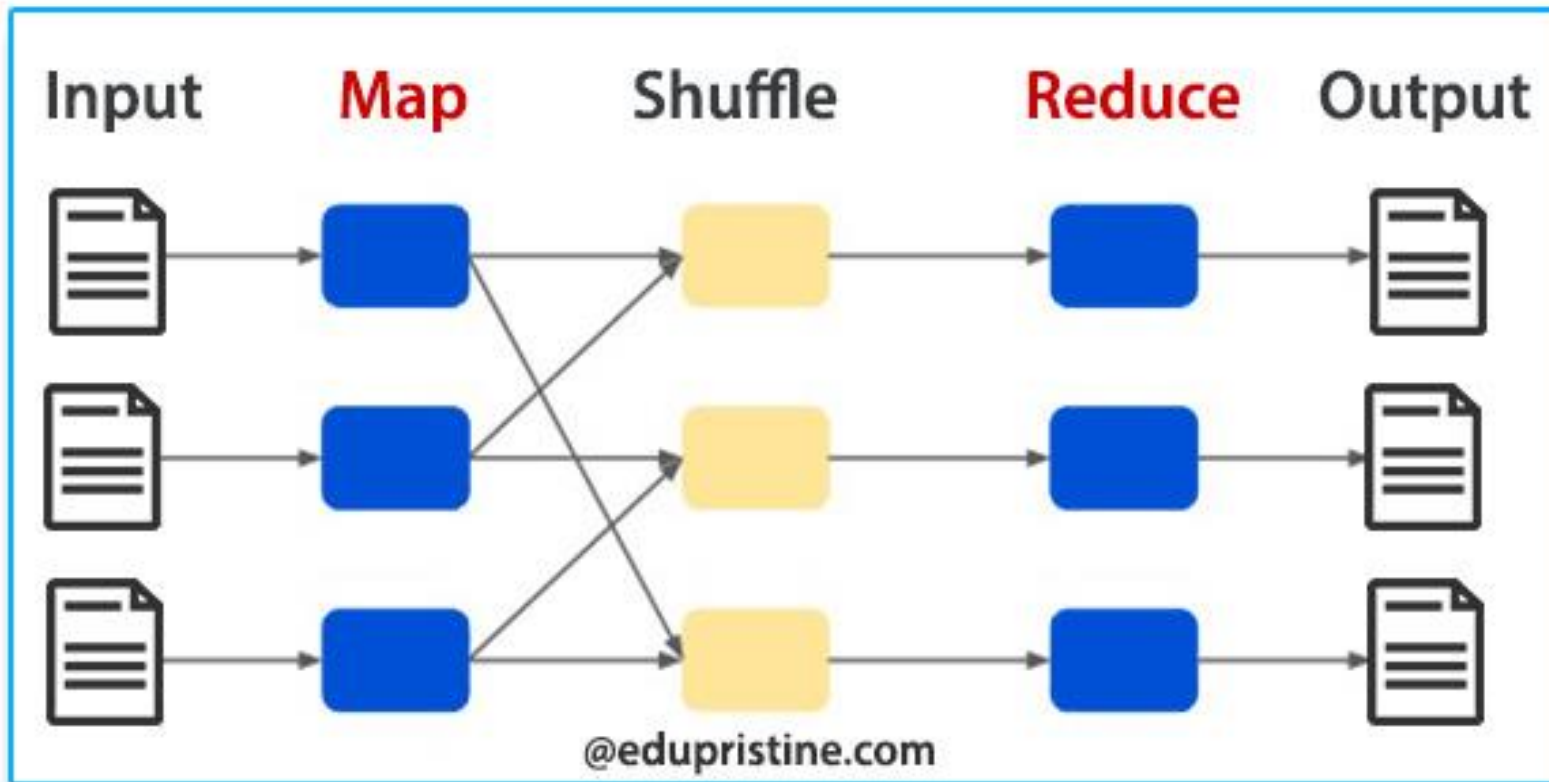
- MapReduce is a foundation model/paradigm for distributed computing using clusters
- MapReduce implementations:
 - Apache Hadoop implements MapReduce
 - Apache Spark implements superset of MapReduce
 - Apache Tez implements MapReduce
 - Amazon Athena: Serverless SQL service
 - Google BigQuery: distributed execution graph and shuffle architecture



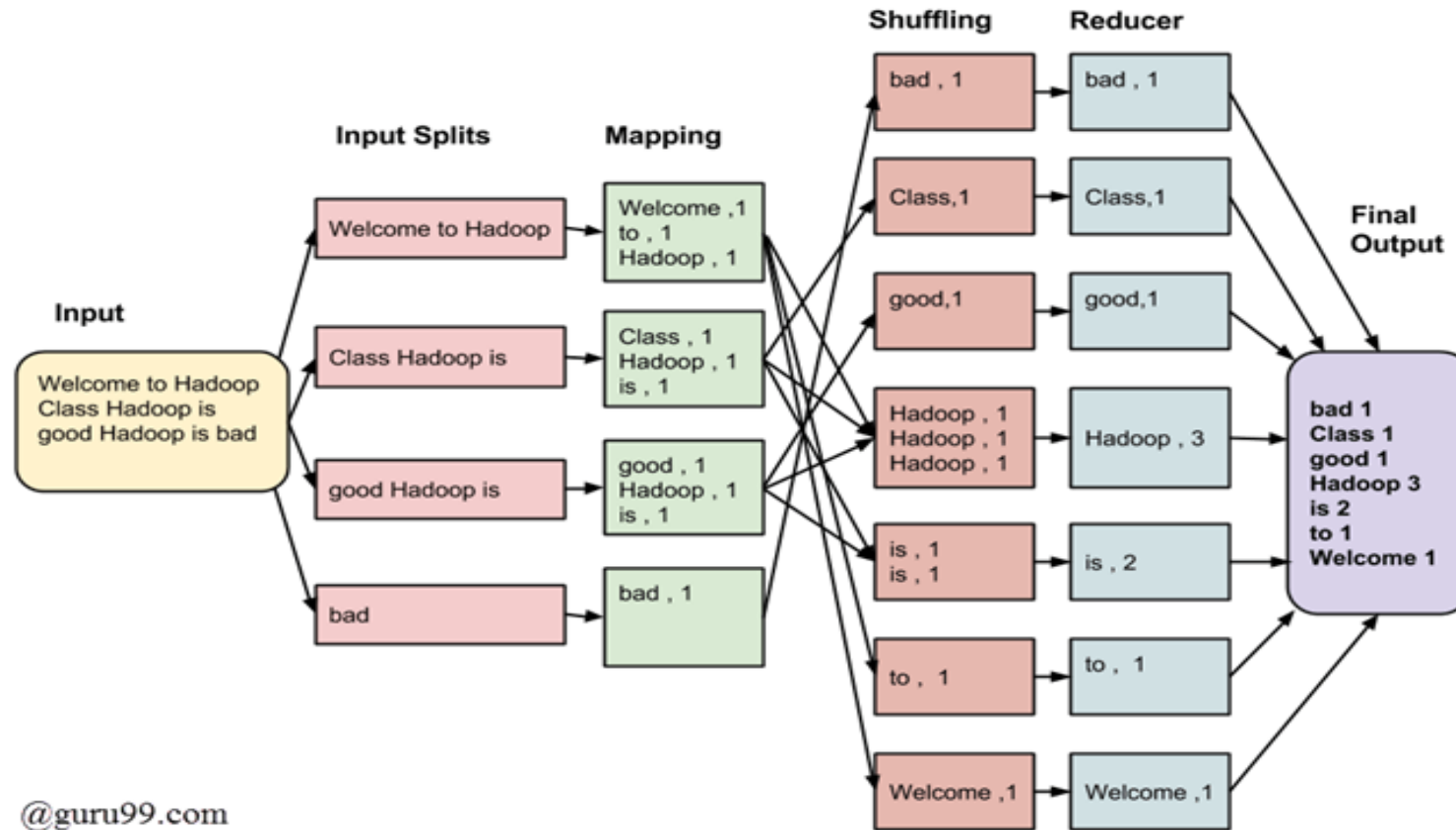
MapReduce Paradigm: High-Level



MapReduce Paradigm



MapReduce Paradigm Example-1

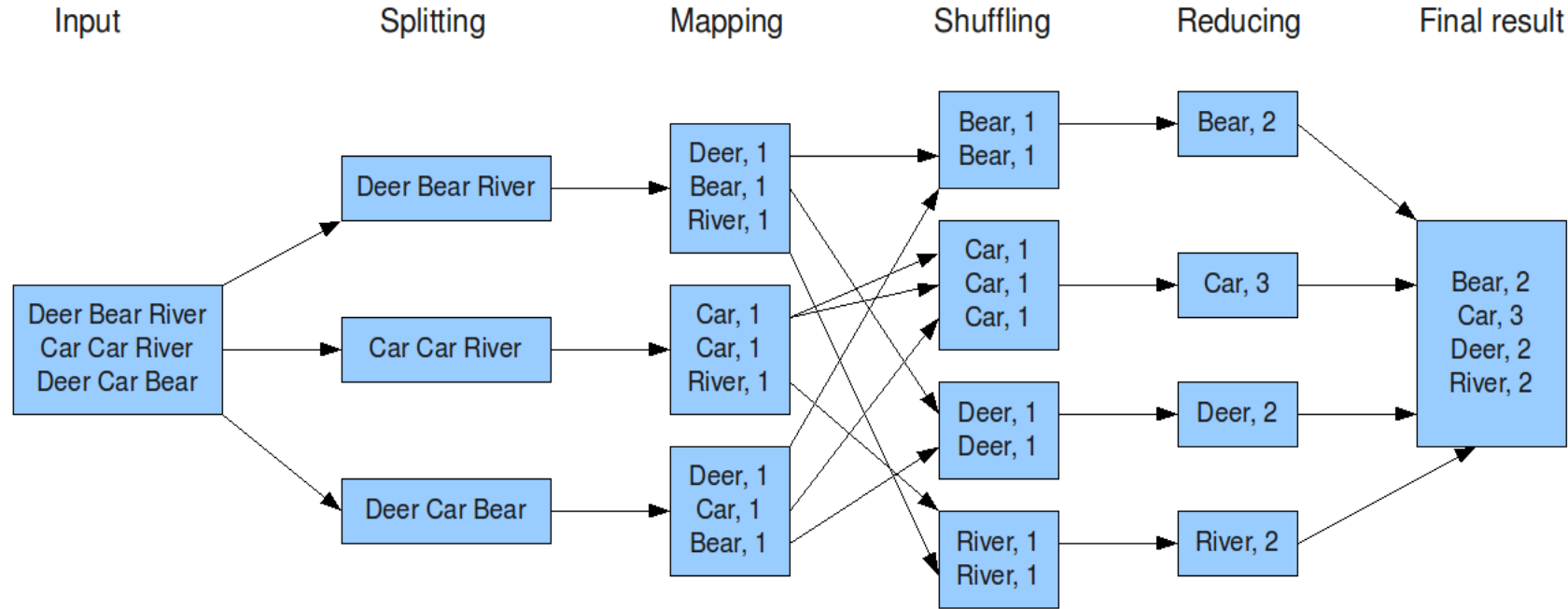


@guru99.com



MapReduce Paradigm Example-2

The overall MapReduce word count process



Solving a Big Data Problem with MapReduce (1)

- Input data is partitioned by a partitioner into chunks and sent to Mappers
- Assume your input is comprised of 80,000,000,000 records, which might be partitioned into 40,000 chunks
 - Number of partitions: 40,000
 - Each partition will have about 2,000,000 records
 - Maximum mapper parallelism can be 40,000 mappers
 - $80,000,000,000 = 40,000 \times 2,000,000$
- A mapper function, $\text{map}(K, V)$ is executed on all partitions in parallel (as much as possible – depends on resources available).
- The $\text{map}(K, V)$ generates any number of (key, value) pairs such as:
- $\{ (K1, V1) , (K2, V2) , \dots \}$. For example, K can be a record number and V can be the actual input record



Solving a Big Data Problem with MapReduce (2)

- All mappers output go to Sort & Shuffle phase, which groups values by unique keys (similar to GROUP BY in SQL).
- **Sort & Shuffle** creates (key, value) pairs as:

(**key_1**, [v1, v2, ...])

(**key_2**, [u1, u2, ...])

...

(**key_n**, [t1, t2, ...])

Where {**key_1**, **key_2**, ... **key_n** }

are the unique keys created by all mappers



Solving a Big Data Problem with MapReduce (3)

- Output of Sort & Shuffle goes to reducers
- A reducer operates on (key, value) pairs where key is in

`{ key_1, key_2, ..., key_n }`

and value is an associated value for the key.

Therefore, the following reducers can be executed in parallel:

```
reduce(key_1, [v1, v2, ...])
```

```
reduce(key_2, [u1, u2, ...])
```

```
...
```

```
reduce(key_n, [t1, t2, ...])
```

- Each reducer can create any number of new (key, value) pairs.



2. Apache Spark

- **Apache Spark** is a multi-language (Java, Python, Scala) engine for executing data engineering, data science, and machine learning on single-node machines or clusters.
- **PySpark** is a Python API for Spark
- **Spark** is a superset of MapReduce
- **Spark** web site: <https://spark.apache.org>



2. Apache Spark Programs

For Apache **Spark** , we will use PySpark and write **actual executable programs**:

```
>>> # spark is an instance of SparkSession object
```

```
>>> rdd = spark.sparkContext
```

```
                .parallelize(range(0, 100))
```

```
>>> rdd.count()
```

```
100
```

```
>>> sum_of_values = rdd.reduce(lambda x, y: x+y)
```

```
>>> sum_of_values
```

```
4950
```



Apache Spark: Components

Streaming

MLlib

For Machine Learning

GraphX

For Graph Computing

**Spark SQL &
DataFrames**

Spark Core API

R

Python

Scala

SQL

Java



Spark run modes

Local Mode Single machine:

- Runs entirely on a single computer or laptop.
- Threads for execution: Uses local threads to simulate worker nodes.
- Testing and development: Best used for debugging and running quick tests.

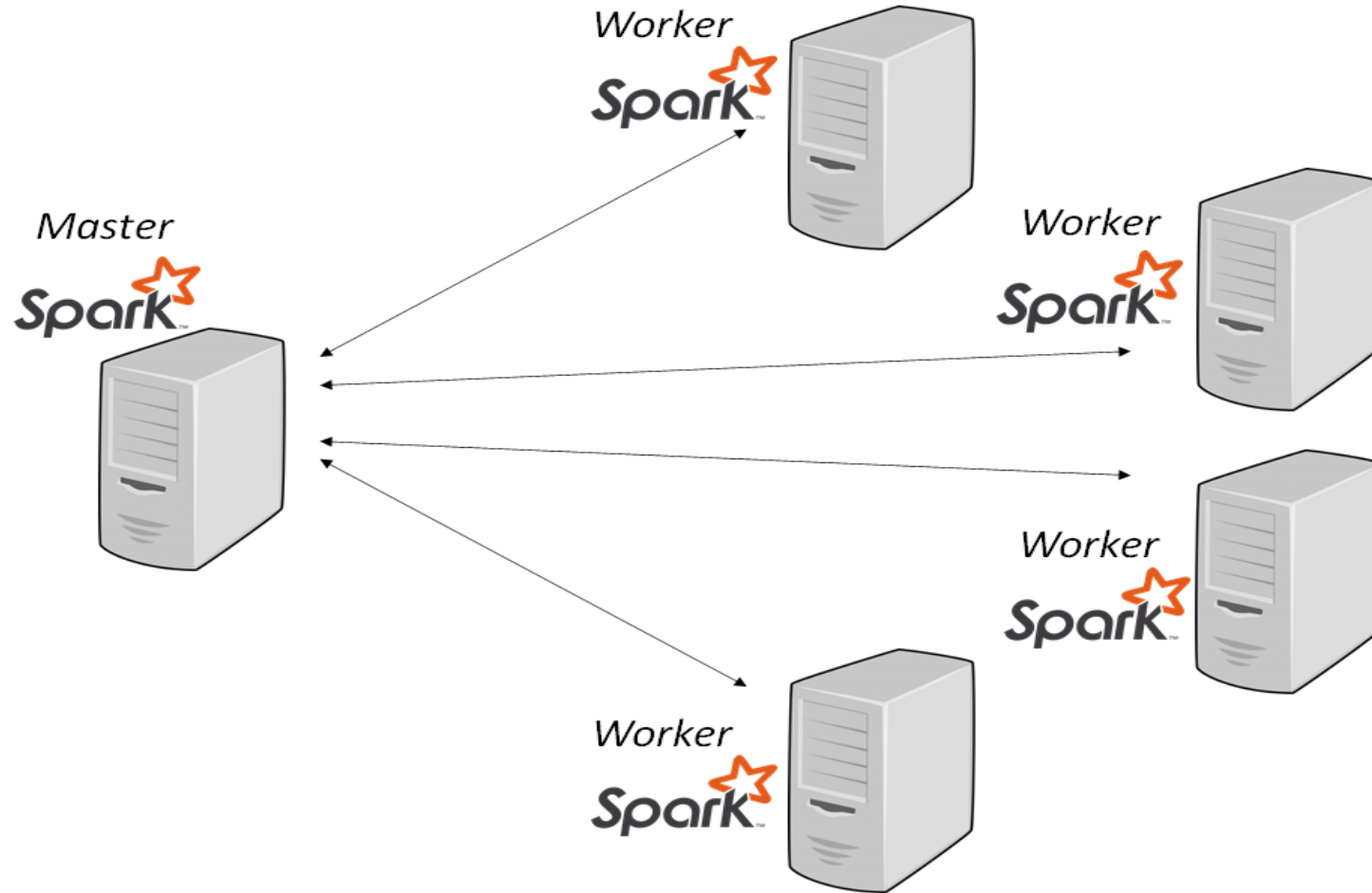
Cluster Modes:

In cluster modes, the main distinction is where the Spark Driver (the program that coordinates the work) runs.

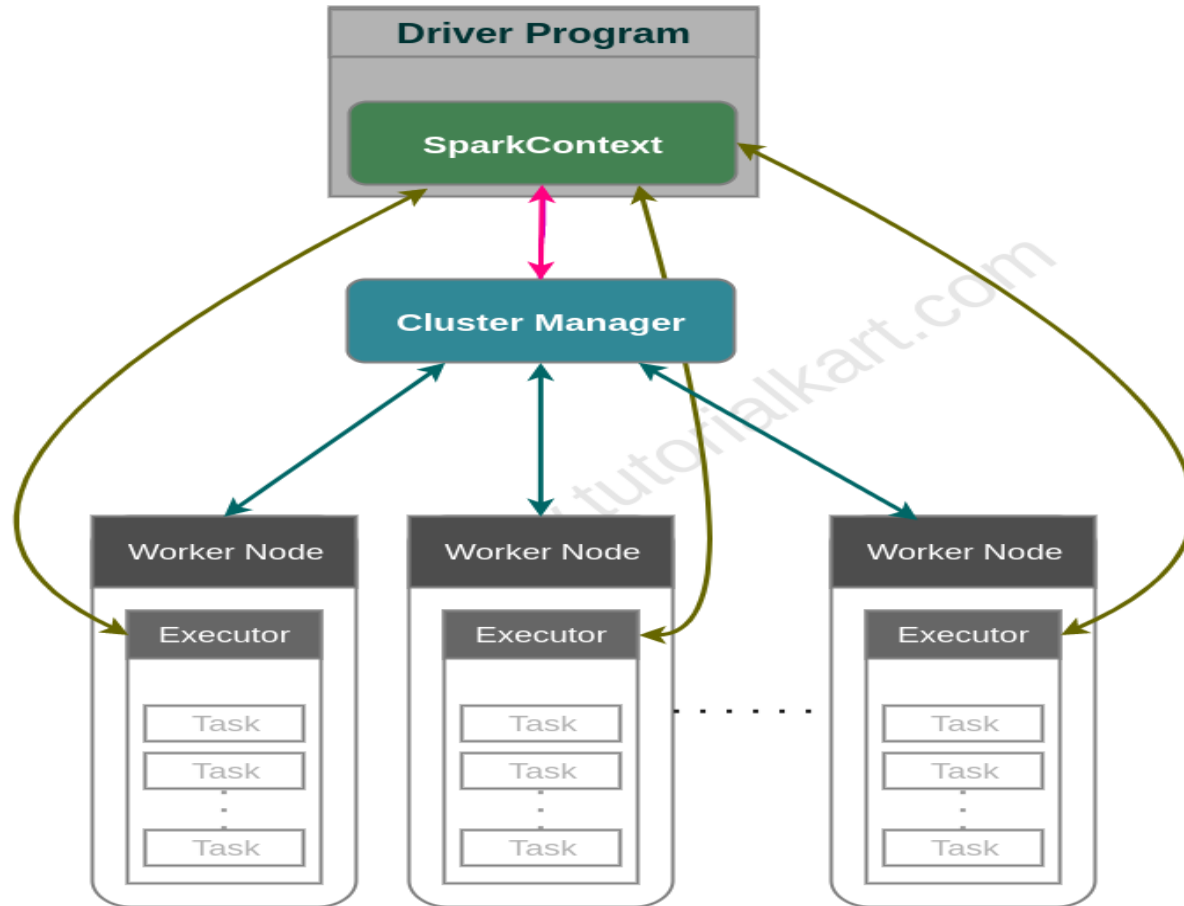
- **Client Mode:** External driver: The driver runs on the machine where the job was submitted.
Interactive use: Best for interactive shells, notebooks, and local debugging. Network dependent: If your local machine disconnects, the job fails.
- **Cluster Mode:** Internal driver: The driver runs inside a worker node within the cluster.
Production standard: Best for scheduled production jobs. Independent: You can close your laptop and the job will keep running.



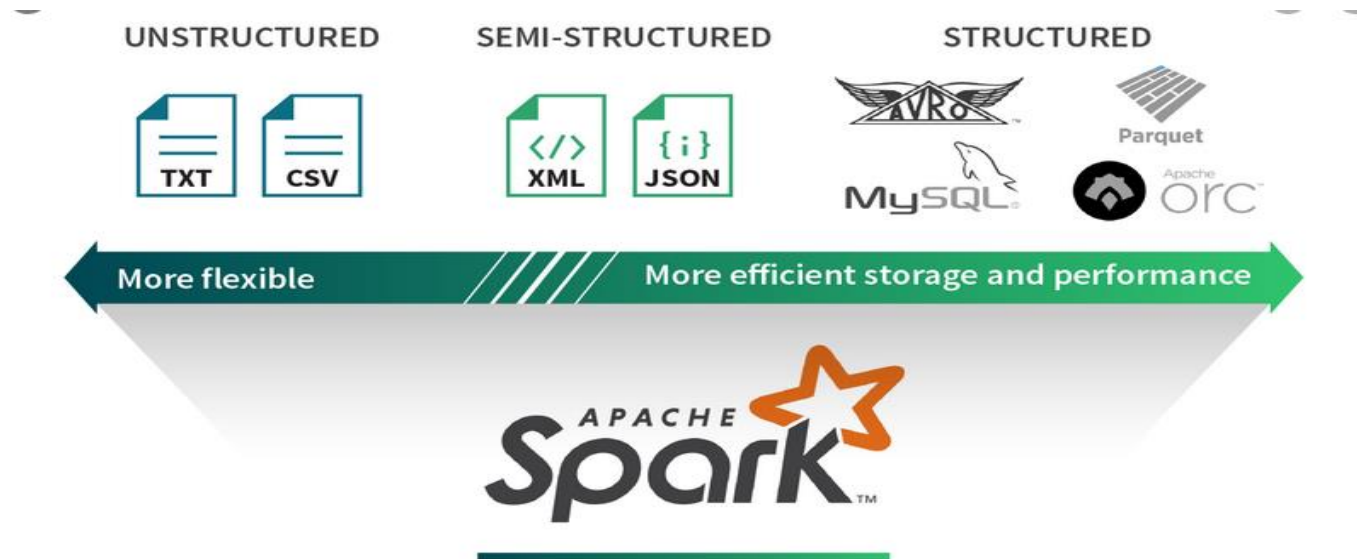
Apache Spark: runs in a cluster



Apache Spark: Cluster Manager



Spark Data Abstractions: Read/Write from/to Many Data Sources



Spark Data Abstractions

- Your data can be represented as an **RDD** and **DataFrame**
- **1. Resilient Distributed Datasets (RDD)**
 - Data is represented as a data type T
(integer, string, tuples, arrays, dictionary, ...)
 - Billions of data points (elements/records)
- **2. DataFrame**
 - Data is represented as a table of rows and named columns
 - Billions of rows of data



Spark Data Abstractions: RDDs

Billions of data elements

Each element: `(String, (Float, Float))`

Element-1:

`(gene-1, (3.0, 4.5))`

Element-2:

`(gene-2, (1.0, 1.5))`

Element-3:

`(gene-1, (2.1, 1.6))`

...

`(gene-8, (3.1, 5.1))`



Spark Data Abstractions: DataFrame

Ways to Create DataFrame in Spark

Hive Data

Csv Data

Json Data

RDBMS Data

XML Data

Parquet Data

Cassandra Data

RDDs

Spark SQL

DataFrame

	Col1	Col2	Col3	
Row 1				
Row 2				
Row 3				
⋮				
⋮				

PySpark DataFrame Example

```
from pyspark.sql import SparkSession
```

```
# 1. Initialize a SparkSession
```

```
spark = SparkSession.builder.appName("demo").getOrCreate()
```

```
# 2. Create a sample DataFrame as df
```

```
data = [ ("Alice", "Sales", 45000),  
         ("Bob", "Engineering", 60000), ...]
```

```
columns = ["name", "department", "salary"]
```

```
df = spark.createDataFrame(data, columns)
```

```
# 3. Register the DataFrame as a Temporary SQL View
```

```
df.createOrReplaceTempView("employees")
```



PySpark DataFrame Example continued...

4. Run an SQL query using spark.sql()

```
query_result = spark.sql("""
    SELECT
        department,
        COUNT(*) as total_employees,
        AVG(salary) as average_salary
    FROM employees
    GROUP BY department
    HAVING average_salary > 42000
    ORDER BY average_salary DESC
    """)
```

5. Display the results

```
query_result.show()
```



PySpark DataFrame Example 2 - 0001

```
# This example introduces key intermediate concepts  
# like aggregation (groupBy), built-in column functions  
# (when, avg), and handling null values (na.fill).
```

```
# 1. Import required libraries
```

```
from pyspark.sql import SparkSession  
from pyspark.sql import functions as F
```

```
# 2. Initialize a SparkSession
```

```
spark = SparkSession.builder.appName("demo").getOrCreate()
```



PySpark DataFrame Example 2 - 0002

```
# 3. Sample e-commerce transaction data
```

```
# (with some missing values)
```

```
data = [ ("User_A", "Electronics", 1200.0, "US"),  
          ("User_B", "Clothing", 45.50, "CA"),  
          ("User_C", "Electronics", None, "US"),  
          ("User_A", "Electronics", 150.0, "US"),  
          ("User_D", "Books", 22.00, "UK"),  
          ("User_B", "Clothing", 80.00, "CA"),  
          ("User_E", "Books", 15.75, None),  
        ]
```

```
schema = ["UserID", "Category", "Amount", "Country"]
```



PySpark DataFrame Example 2 - 0003

4. Create a DataFrame

```
df = spark.createDataFrame(data, schema=schema)
```

```
print("--- Raw Data ---")
```

```
df.show()
```

```
--- Raw Data ---
```

```
+-----+-----+-----+-----+
|UserID|  Category|Amount|Country|
+-----+-----+-----+-----+
|User_A|Electronics|1200.0|    US|
|User_B|  Clothing|  45.5|    CA|
|User_C|Electronics|  null|    US|
|User_A|Electronics| 150.0|    US|
|User_D|      Books|  22.0|    UK|
|User_B|  Clothing|  80.0|    CA|
|User_E|      Books| 15.75|  null|
+-----+-----+-----+-----+
```



PySpark DataFrame Example 2 - 0004

```
# 5. Data Cleaning: Handle missing values
```

```
# Fill missing Amounts with 0.0, and
```

```
# Fill missing Countries with 'Unknown'
```

```
df_cleaned = df.na.fill({"Amount": 0.0, "Country": "Unknown"})
```

```
# 6. Data Transformation: Add a conditional "Customer_Segment" column
```

```
# High spenders (> $500), Medium spenders (> $0), and Inactive ($0)
```

```
df_transformed = df_cleaned.withColumn( "Segment",
```

```
F.when(F.col("Amount") > 500, "High")
```

```
.when(F.col("Amount") > 0, "Medium")
```

```
.otherwise("Inactive"))
```

```
print("--- Cleaned and Transformed Data ---")
```

```
df_transformed.show()
```



PySpark DataFrame Example 2 - 0005

--- Cleaned and Transformed Data ---

```
+-----+-----+-----+-----+-----+
|UserID|  Category|Amount|Country|Segment|
+-----+-----+-----+-----+-----+
|User_A|Electronics|1200.0|    US|   High|
|User_B|  Clothing|  45.5|    CA| Medium|
|User_C|Electronics|   0.0|    US|Inactive|
|User_A|Electronics| 150.0|    US| Medium|
|User_D|      Books|  22.0|    UK| Medium|
|User_B|  Clothing|  80.0|    CA| Medium|
|User_E|      Books| 15.75|Unknown| Medium|
+-----+-----+-----+-----+-----+
```



PySpark DataFrame Example 2 - 0006

7. Data Aggregation: Calculate metrics by Category

Total spend, average spend, and distinct customer count per category

```
df_metrics = df_transformed.groupBy("Category").agg(  
    F.sum("Amount").alias("Total_Revenue"),  
    F.round(F.avg("Amount")).alias("Average_Spend"),  
    F.countDistinct("UserID").alias("Unique_Customers")  
)orderBy(F.desc("Total_Revenue"))
```

```
print("--- Aggregated Category Metrics ---")
```

```
df_metrics.show()
```



PySpark DataFrame Example 2 - 0007

--- Aggregated Category Metrics ---

Category	Total_Revenue	Average_Spend	Unique_Customers
Electronics	1350.0	450.0	2
Clothing	125.5	62.75	1
Books	37.75	18.88	2



3. Main Course Components

Serverless SQL Access to Big Data (15%)

- Amazon Athena
- Google BigQuery
- Snowflake



Amazon Athena

- Interactive query service
- Serverless. Zero infrastructure. Zero administration.
- Put your data in S3
- Access your data by SQL
- Pay by query
- Fast performance



Amazon Athena

